Lab Assignment 5

Nested Loops

CSE110: Programming Language I

No of Tasks	Points to Score
12	**120**

Submit the coding tasks (Task 1 - 8) on buX and the handwritten tasks (Task 9 - 12) to your ST / Lab Instructors in the beginning of the next lab class.

1. Write a Java program to take a positive integer ***N*** (where N > 0) as user input and print the **first *N* prime numbers starting from 2**. Your code should check all the positive integers starting from 2 and determine whether they are prime or not until ***N*** prime numbers are found.

   <u>Sample Input 1:</u>
   5
   <u>Sample Output 1:</u>
   2
   3
   5
   7
   11
   <u>Sample Input 2:</u>
   7
   <u>Sample Output 2:</u>
   2
   3
   5
   7
   11
   13
   17

2. Write a Java code of a program that reads the value of N (where N > 0) from the user and calculates the value of y if the expression of y is as follows:

$$y = -(1) - (1 + 2) - (1 + 2 + 3) - \ldots - (1 + 2 + 3 + \ldots + N)$$

   Sample Input:
   The value of N: 2
   Sample Output:
   The value of y: -4

   Sample Input:
   The value of N: 4
   Sample Output:
   The value of y: -20

3. Write a Java program that will keep taking even positive integer numbers as inputs from the user and print the number of divisors of those numbers until it gets an odd number and then stops.

   Sample Input & Output: (The purple numbers are input)
   Enter Number: 44
   44 has 6 divisors
   Enter Number: 30
   30 has 8 divisors
   Enter Number: 8
   8 has 4 divisors
   Enter Number: 4
   4 has 3 divisors
   Enter Number: 6
   6 has 4 divisors
   Enter Number: 20
   20 has 6 divisors
   Enter Number: 24
   24 has 8 divisors
   Enter Number: 5

4. Read an integer N that is the number of test cases that follow. Each test case contains two integers X and Y. Print one output line for each test case that the sum of Y odd numbers from X including it if is the case. For example:
   For the input 4 5, the output must be 45, that is: 5 + 7 + 9 + 11 + 13
   For the input 7 4, the output must be 40, that is: 7 + 9 + 11 + 13

Sample Input	**Sample Output**
2 4 3 11 2	21 24
Explanation: Here, the 2 means there are two test cases. For each test case you have to take two inputs (X, Y) and print the sum of Y odd numbers starting from X.	

5. Take the length and width of a **rectangle** from the user and create the rectangle according to the output below. Your output should match the specified output.

Sample Input #1	**Sample Input #2**
4 6	3 5
Output 1 2 3 4 1 2 3 4 1 2 3 4 1 2 3 4 1 2 3 4 1 2 3 4	**Output** 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3

6. Take the height of a **right-justified right triangle** from the user and create the triangle according to the output below. Your output should match the specified output.

Sample Input #1	**Sample Input #2**
4	3

Output
  1
  1 2
  1 2 3
1 2 3 4
	Output
	  1
	  1 2
	1 2 3

7. Take the height of an **isosceles triangle** from the user and create the triangle according to the output below. Your output should match the specified output.

Sample Input #1	**Sample Input #2**
4	3

Output	**Output**
1 1 2 3 1 2 3 4 5 1 2 3 4 5 6 7	1 1 2 3 1 2 3 4 5

8. Write a Java program that will ask for a range (a starting number and an ending number) from the user and print all the Armstrong numbers between that range.

   [Armstrong Number: *An Armstrong number is a number whose sum of digits raised to the power the number of digits equals to that number.*

   For example, 371 is an Armstrong number because $3^3 + 7^3 + 1^3 = 371$*, here the total number of digits in 371 is 3* ***]***

   <u>Sample Input 1:</u>
   Start: 300
   End: 500

   <u>Sample Output 1:</u>
   Armstrong numbers:
   370 ->
   371
   407
   <u>Sample Input 2:</u>
   Start: 100
   End: 200
   <u>Sample Output 2:</u>
   Armstrong numbers:
   153

9. Trace the following code, create a tracing table and write the outputs.

```
public class T1{
   public static void main(String args[]){
     int x = 0, y = 0;
     int sum = 0;
     while (x < 4){
       y = x - 3;
       while (y < 3){
           sum = (sum % 3) + x - y * 3 ;
           System.out.println(sum);
           y = y + 1;
       }
       if (x > 5){
           x++;
       }
       else{
           x += 2;
       }
     }
   }
}
```

10. Trace the following code, create a tracing table and write the outputs.

```
public class T2 {
  public static void main(String args[]) {
      int x = 0, i = 0, sum = 0;
      i = 1;
      x = 2;
      sum = 0;
      while (i < 20){
         x = x + i;
         sum = sum + x + 1;
         System.out.println(sum);
         if (x > 5){
           i += 2;
         }
         else {
           i += 3;
         }
      }
      sum = sum + i;
      System.out.println(sum);
   }
}
```

11. Trace the following code, create a tracing table and write the outputs.

```
public class T3
{
  public static void main(String args[])
  {
     int x = 0, y = 0;
     int sum = 0;
     while (x < 10){
       y = x - 3;
       y = 40;
       while (y > 22){
         if ((sum > 30) && (sum < 40)){
            sum = sum + x * 2 ;
         }
         else if ((sum > 40) && (sum < 50)){
            sum = sum + x * 3;
         }
         else {
            sum = sum + 23;
         }
         System.out.println(sum);
         y = y - 10;
       }
       x += 2;
     }
  }
}
```

12. Trace the following code, create a tracing table and write the outputs

```
public class T4
{
  public static void main(String args[])
  {
      int x = 0, p = 0, sum = 0;
      p = 1;
      x = 2;
      double q;
      sum = 0;
      while (p < 12){
         q =  x + p-(sum+7/3)/3.0%2 ;
         sum = sum + (++x) + (int)q;
         System.out.println(sum++);
         if (x > 5){
            p += 4/2;
         }
         else {
            p += 3%1;
         }
      }
      sum = sum + p;
      System.out.println(sum);
  }
}
```